III B.Sc., Computer Science
Semester-V

Application Development using Python Practical's
2025-2026



DEPARTMENT OF COMPUTER SCIENCE

St. Joseph's Degree College
Sukesula Road
Kurnool.

1. Write a menu driven program to convert the given temperature from Fahrenheit to Celsius and vice versa depending upon user's choice.

Source Code:

```
print('1. Celsius to Fahrenheit.')
print('2. Fahrenheit to Celsius.')
choice = int(input('Enter your choice: '))
if choice == 1:
  c = float(input('Enter temperature in celsius: '))
  f = 9 * c / 5 + 32
  print('Temperature in fahrenheit scale:', f)
elif choice == 2:
  f = float(input('Enter temperature in fahrenheit: '))
  c = 5 * (f - 32) / 9
  print('Temperature in celsius scale:', c)
else:
  print('Invalid choice.')
```

2. Write a python program to calculate total marks, percentage and grade of a student. Marks obtained in each of the three subjects are to be input by the user. Assign grades according to the following criteria :

Grade A: Percentage >=80
Grade B: Percentage>=70 and 80
Grade C: Percentage>=60 and <70
Grade D: Percentage>=40 and <60
Grade E: Percentage<40

Source Code:

```
sno = int(input("Enter Student No:"))
name = input("Enter Student Name:")
subject_1 = int(input("Enter Subject1 Marks:"))
subject_2 = int(input("Enter Subject2 Marks:"))
subject_3 = int(input("Enter Subject3 Marks:"))
subject_4 = int(input("Enter Subject4 Marks:"))
subject_5 = int(input("Enter Subject5 Marks:"))
total, average, percentage, grade = None, None, None, None

total = subject_1 + subject_2 + subject_3 + subject_4 + subject_5
average = total / 5.0
percentage = (total / 500.0) * 100

if average >= 80:
   grade = 'A'
elif average >= 70 and average < 80:
   grade = 'B'
elif average >= 60 and average < 70:
   grade = 'C'
elif average >= 40 and average < 60:
   grade = 'D'
else:
   grade = 'E'

print("\nStudent Details")
print("Student No:", sno)
print("Student Name:" ,name)
print ("The Total marks is:", total)
print ("The Percentage is:", percentage)
print ("The Grade is:", grade)
```

3. Demonstrate various methods of Sequence Data Types

Source Code:

```
print("Various Methods of Sequence Data Types\n")
my_string = "Welcome to Python Programming"

# List: a collection of different types of data
my_list = [25, "Hello", 2.5, True]

# Tuple: a fixed collection of values
my_tuple = ("Apple", "Banana", "Cherry")

# Create a range object
my_range = range(1, 10, 2)

# Create a byte sequence
my_bytes = bytes([65, 66, 67])

 # Create a byte array
my_byte_array = bytearray([65, 66, 67])
my_byte_array[0] = 68

# Printing all Sequences

print("String:", my_string)
print("List:", my_list)
print("Tuple:", my_tuple)
print("Range:", my_range)
print("Bytes:", my_bytes)


# Accessing Elements

print("\nAccessing elements:")
print("First word in string:", my_string[0])
print("Second item in list:", my_list[1])
print("Last fruit in tuple:", my_tuple[-1])
print("Range of List:", list(my_range))
print("Byte Sequences:", my_bytes[:2])
print("Byte Arrays:", my_byte_array)
```

4. Write a python program to display the first n terms of Fibonacci series

Source Code:

```
num = int(input("Enter How many Terms you want:"))
t1 = 0
t2 = 1
i = 1
print("Fibinocci Series")
while i<=num:
   print (t1)
   nextTerm = t1 + t2
   t1 = t2
   t2 = nextTerm
   i = i + 1
```

5. Write a Python program to read last 5 lines of a file.

Source Code:

```
filename = "D:\sample.txt" # Replace with your file name

with open(filename, "r") as file:
   lines = file.readlines()

# Get the last 5 lines last_five = lines[-5:]

# Print them
for line in lines:
   print(line, end='')
```

6. Write a python program to calculate the sum and product of two compatible matrices.

Source Code:

```
rows = int(input('Enter number of rows: '))
cols  = int(input('Enter number of column: '))

print() # for new line
print('Enter values for matrix A')
matrix_A = [[int(input("Enter any value:"))
for j in range(cols)]
 for i in range(rows) ]

print() # for new line
print('Enter values for matrix B ')
matrix_B = [[int(input("Enter any value:"))
for j in range(cols)]
for i in range(rows) ]

print() #for new line

# resultant matrix (matrix that store answer and intially it is Zero)
result = [[0 for j in range(cols)]
for i in range(rows)]

# addition
for i in range(rows):
    for j in range(cols):
        result[i][j] = matrix_A[i][j] + matrix_B[i][j]

print() #for new line
print('Addition of Matrix-A and Matrix-B is :')
for i in result:
    print(i)

# Multiplication
for i in range(rows):
    for j in range(cols):
        for k in range(cols):
            result[i][j] = matrix_A[i][k] * matrix_B[k][j]

print() #for new line
print('Multiplication of Matrix-A and Matrix-B is :')
for i in result:
    print(i)
```

7. Write a program to implement exception handling.

Source Code:

```
def divide_numbers(numerator, denominator):

    try:
        result = numerator / denominator
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except ValueError as e
        print(f"Error: Invalid input type - {e}")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return None
    else:
        print(f"Division successful. Result: {result}")
        return result
    finally:
       print("Division attempt complete.")

# --- Test Cases ---
# Case 1: Successful division
print("--- Test Case 1: Valid Division ---")
divide_numbers(10, 2)
print("\n")

# Case 2: ZeroDivisionError
print("--- Test Case 2: Division by zero ---")
divide_numbers(10, 0)
print("\n")

# Case 3: ValueError (invalid type for denominator)
print("--- Test Case 3: Invalid input type ---")
divide_numbers(10, "abc")
print("\n")
```

8. Write a program to implement Multithreading

Source Code:

```
import threading
import time
# Function for thread 1
def print_numbers():
    for i in range(1, 6):
        print(f"Number: {i}")
        time.sleep(1)

# Function for thread 2
def print_letters():
    for letter in ['A', 'B', 'C', 'D', 'E']:
        print(f"Letter: {letter}")
        time.sleep(1)
print()
# Create threads
thread1 = threading.Thread(target=print_numbers)
thread2 = threading.Thread(target=print_letters)

# Start threads
thread1.start()
thread2.start()

# Wait for both threads to finish
thread1.join()
thread2.join()

print("Finished executing both threads.")
```

9. Develop a Python GUI calculator using Tkinter

Source Code:

```
import tkinter as tk

def click(event):
    text = event.widget.cget("text")
    if text == "=":
        try:
            result = str(eval(str(entry.get())))
            entry.delete(0, tk.END)
            entry.insert(0, result)
        except:
            entry.delete(0, tk.END)
            entry.insert(0, "Error")
    elif text == "C":
        entry.delete(0, tk.END)
    else:
        entry.insert(tk.END, text)

root = tk.Tk()
root.title("Calculator")

entry = tk.Entry(root, font="Arial 20")
entry.grid(row=0, column=0, columnspan=4, ipadx=8, ipady=8)

buttons = [
    ["7", "8", "9", "/"],
    ["4", "5", "6", "*"],
    ["1", "2", "3", "-"],
    ["C", "0", "=", "+"]
]

for i, row in enumerate(buttons):
    for j, text in enumerate(row):
        btn = tk.Button(root, text=text, font="Arial 18", width=5, height=2)
        btn.grid(row=i+1, column=j, padx=3, pady=3)
        btn.bind("<Button-1>", click)

root.mainloop()
```

10. Design a simple database application that stores the records and retrieve the same

Source Code:

```
import tkinter as tk
import sqlite3

# DB setup
conn = sqlite3.connect('data.db')
c = conn.cursor()
c.execute("CREATE TABLE IF NOT EXISTS records (name TEXT, age INTEGER)")
conn.commit()

# Functions
def add():
    c.execute("INSERT INTO records VALUES (?, ?)", (e1.get(), e2.get()))
    conn.commit()
    e1.delete(0, tk.END)
    e2.delete(0, tk.END)

def view():
    c.execute("SELECT * FROM records")
    rows = c.fetchall()
    t.delete('1.0', tk.END)
    for row in rows:
        t.insert(tk.END, f"{row[0]}, {row[1]}\n")

# GUI
root = tk.Tk()
root.title("Simple DB App")
tk.Label(root, text="Name").grid(row=0, column=0)
e1 = tk.Entry(root);
e1.grid(row=0, column=1)
tk.Label(root, text="Age").grid(row=1, column=0)
e2 = tk.Entry(root);
e2.grid(row=1, column=1)

tk.Button(root, text="Add", command=add).grid(row=2, column=0, pady=5)
tk.Button(root, text="View", command=view).grid(row=2, column=1, pady=5)

t = tk.Text(root, height=10, width=40)
t.grid(row=3, column=0, columnspan=2, pady=10)
root.mainloop()
conn.close()
```